Leetcode

1 Longest Substring Without Repeating Characters

Brief Problem Description:
Input: s = "abcabcbb" Output: 3 Explanation: The answer is "abc", with the length of 3.

Brute Force

Check all substrings with n^2 loop, for each substring call function to check uniqueness by checking each character. Data structures used: - Hashset

Time Complexity:
Time: $O(n^3)$ (worst case- total unique string) Space: O(k) where k is min (n, alphabet size)

Sliding window 1

i=0,j=0, j is moved to right, hashmap store with count until j repeats (count>1), in which case i is moved to right(and deleted from hashmap) until count is 1.

Time Complexity:
Time: O(2n) worst case each charc will be visited twice ex:- abcabcbb Space: O(k) where k is min of alpha size and n

Sliding Window Optimized

Why move i little by little when you can just move it directly to the next position of repeated charc? If j' is repeat index of i,j window, then move i to j'+1. Using hashmap to store char− >its nxt ind, hmap.containskey to check existance

Time Complexity:
O(n) space: same

---

2 Valid Sudoku

Brief Problem Description:
A valid sudoku board should satisfy three conditions: (1) each row, (2) each column, and (3) each box has no duplicate numbers.

Brute Force

Three pass approach, one for row, one for column, one for the box (so you pass the index where there's a number to these three functions)

Time Complexity:
Time Complexity: O(1) Space Complexity: O(1) (since the space used by the HashSets in each method is limited to at most 9 elements)

Approach 1

The first approach description goes here.

Time Complexity:
A description of the time complexity of the first approach goes here.

Approach 2

one pass Create dictionaries to keep track of numbers seen in rows, columns, and 3x3 box. Iterate Through the Board Row Check: If the number is not in the corresponding row list, add it. If the number is already in the row list, return False (invalid Sudoku). Column Check: If the number is not in the corresponding column list, add it. If the number is already in the column list, return False (invalid Sudoku). box Check: Determine the box index for the current cell. If the number is not in the corresponding box list, add it. If the number is already in the box list, return False (invalid Sudoku). Return True if no duplicates in box, Column, Row Else False, if Duplicates Code

```
class Solution(object):
    def isValidSudoku(self, board):
        rows = [[] for _ in range(9)]
        columns = [[] for _ in range(9)]
        box = [[] for _ in range(9)]

        for i in range(9):
```

```
            for j in range(9):
                num = board[i][j]
                if num != ".":
                    box_index = (i // 3) * 3 + (j // 3)
                    if num in rows[i] or num in columns[j] or
                        num in box[box_index]:
                        return False
                    rows[i].append(num)
                    columns[j].append(num)
                    box[box_index].append(num)

        return True
```

Time Complexity:
Time Complexity: O(1) Space Complexity: O(1) (since the space used by the HashSets in each method is limited to at most 9 elements)

3 Three Sum

Brief Problem Description:
Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0. Notice that the solution set must not contain duplicate triplets.

Brute Force

n^3 approach naturally
Time Complexity:
$O(n^3)$

Two pointers

Sort the input array nums. Iterate through the array: If the current value is greater than zero, break from the loop. Remaining values cannot sum to zero. If the current value is the same as the one before, skip it. Otherwise, call twoSumII for the current position i. For twoSumII function: Set the low pointer lo to i + 1, and high pointer hi to the last index. While lo pointer is smaller than hi: If the sum of nums[i] + nums[lo] + nums[hi] is less than zero, increment lo. If the sum is greater than zero, decrement hi. Otherwise, we found a triplet: Add it to the result res. Decrement hi and increment lo. Increment lo while the next value is the same as before to avoid duplicates in the result. Return the result res.

Time Complexity:
• Time Complexity: $O(n^2)$. twoSumII is O(n), and we call it n times. • Space Complexity: from O(logn) to O(n), depending on the implementation of the sorting algorithm.

Hashset

Sort the input array nums. Iterate through the array: If the current value is greater than zero, break from the loop. Remaining values cannot sum to zero. If the current value is the same as the one before, skip it. Otherwise, call twoSum for the current position i. For twoSum function: For each index j ¿ i in nums: Compute complement value as -nums[i] - nums[j]. If complement exists in hashset seen: We found a triplet - add it to the result res. Increment j while the next value is the same as before to avoid duplicates in the result. Add nums[j] to hashset seen. Return the result res.

Time Complexity:
• Time Complexity: $O(n^2)$. twoSum is O(n), and we call it n times. • Space Complexity: O(n) for the hashset.

---

4 trapping rainwater

Brief Problem Description:
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1] Output: 6 Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped. (first,last index can't trap rain cuz nothing before, so not cnsdrd whle clctng)

4.1 Brute Force

loop through array, for each i iterate forward, backward calculating leftmax, rightmax for each i everytime. ans+ =min(leftmax, rightmax)- height for each i

Time Complexity:
Time: $O(n^2)$ Space: O(1)

4.2 DP

instead for calculating everytime, do two passes to calculate leftmax, rightmax arrays by left,rightmax[0] = height[0], height[size-1] obvsly leftmax[i] = Math.max(height[i], leftmax[i - 1]); (1 to len) rightmax[i] = Math.max(height[i], rightmax[i + 1]); (size-1 to 0)

Time Complexity:
O(n) both

4.3 Stack

one pass stack approach where, you keep adding to stack if lesser or equal to previous one, if suddenly greater than prev one, that means we have like a pothole, so if curr¿ hieght[st. top], we store st.top in top, pop the st.top, st.top is now prev top, something like this st.top, top, curr resp (3,2,4) and we just calc ans+= distance b/w curr nd st.top * min(st.top,curr)-top

Time Complexity:
O(n) both.

4.4 2 pointers

so basically, the same concept to check the dip, we do it using two pointers from left and right, so wherever dip occurs first i.e if left less than right, we dive in to check there, calc left max = max(leftmax, left), ans+=leftmax-left (so here if left was a dip from leftmax then diff added to answer, else leftmax updated and left-leftmax=0=ans) same for right (if right less than left)

Time Complexity:
time:O(n), space: O(1)

---

5 Longest Consecutive sequence binary tree

Brief Problem Description:
Given the root of a binary tree, return the length of the longest consecutive sequence path. A consecutive sequence path is a path where the values increase by one along the path. Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

5.1 top down/preorder

traverse the binary tree in preorder(CLR), checking each node is one more than its parent, then dfs to the left, right

```
1 private int maxLength = 0;
2 public int longestConsecutive(TreeNode root) {
3     dfs(root, null, 0);
4     return maxLength;
5 }
6
```

```
7  private void dfs(TreeNode p, TreeNode parent, int length) {
8      if (p == null) return;
9      length = (parent != null && p.val == parent.val + 1) ?
           length + 1 : 1;
10     maxLength = Math.max(maxLength, length);
11     dfs(p.left, p, length);
12     dfs(p.right, p, length);
13 }
```

Time Complexity:
Time: $O(n)$ Space: O(1)

6 Binary subarrays wih sum

Brief Problem Description:
We are given a binary array nums and an integer goal. The task is to find the number of non-empty subarrays in the given binary array where the sum of elements in the subarray equals the specified goal. Key Observations: The array contains only binary values (0 or 1). The goal is to find subarrays with a specific sum. The subarrays should be non-empty and contiguous. Consider the given example with nums = [1,0,1,0,1] and goal = 2: Output: 4 Explanation: The 4 subarrays are bolded and underlined below: [1,0,1,0,1] [1,0,1,0,1] [1,0,1,0,1] [1,0,1,0,1] Note that all these subarrays are contiguous parts of the given array, and the count of such subarrays is the output.

6.1 Prefix Sum

A prefix sum up to index i is the sum of all elements from the start of the array up to i If you have the prefix sum up to index i and you know the prefix sum up to some earlier index j, the sum of the subarray from j+1 to i can be found by subtracting the prefix sum up to j from the prefix sum up to i

Suppose we want to find subarrays that sum to goal. Let current-sum be the prefix sum up to the current index. Using Prefix Sums to Find Subarrays: We want to find if there's any subarray ending at the current index that sums to goal. For a subarray from index j+1 to i to sum to goal, the following must be true: current-sum - prefix-sum up to j = goal prefix-sum up to j = current-sum - goal Checking and Counting: Therefore, if current-sum - goal exists in the dictionary freq (which keeps track of the frequency of all prefix sums encountered so far), it means there are subarrays ending at the current index that sum to goal.The value associated with current-sum - goal in freq gives the number of such subarrays.

iterate through the array to keep updating current sum, keeping a map to store frequencies of all current sums, if current sum meets goal then increase count (subarray found) if freq map has something with current sum - goal(eg. 4-2 =2, so subarrays with curr sum 2) , then add it to count as well as they are potential subarrays.

Time Complexity:

Time: $O(n)$ Space: O(n)

6.2 Sliding Window

calculating the total count of subarrays with sums less than or equal to the goal using the functionslidingWindowAtMost(nums, goal), we need to isolate the subarrays that strictly meet the target goal.

This can be achieved by subtracting the total count of subarrays with sums less than the goal (slidingWindowAtMost(nums, goal - 1)) from the total count obtained earlier. By subtracting the latter from the former, we remove the subarrays that don't reach the goal and are left with only the subarrays that have a sum exactly equal to the goal.

Time Complexity:

O(n) both

6.3 Stack

one pass stack approach where, you keep adding to stack if lesser or equal to previous one, if suddenly greater than prev one, that means we have like a pothole, so if curr¿ hieght[st. top], we store st.top in top, pop the st.top, st.top is now prev top, something like this st.top, top, curr resp (3,2,4) and we just calc ans+= distance b/w curr nd st.top * min(st.top,curr)-top

Time Complexity:

O(n) both.

6.4 2 pointers

so basically, the same concept to check the dip, we do it using two pointers from left and right, so wherever dip occurs first i.e if left less than right, we dive in to check there, calc left max = max(leftmax, left), ans+=leftmax-left (so here if left was a dip from leftmax then diff added to answer, else leftmax updated and left-leftmax=0=ans) same for right (if right less than left)

Time Complexity:

time:O(n), space: O(1)

6.5 Related problem - subarrays with k odd numbers

so basically, the same concept to check the dip, we do it using two pointers from left and right, so wherever dip occurs first i.e if left less than right, we dive in to check there, calc left max = max(leftmax, left), ans+=leftmax-left (so here if left was a dip from leftmax then diff added to answer, else leftmax updated and left-leftmax=0=ans) same for right (if right less than left)